

Aula 05:

Arquitetura MVC

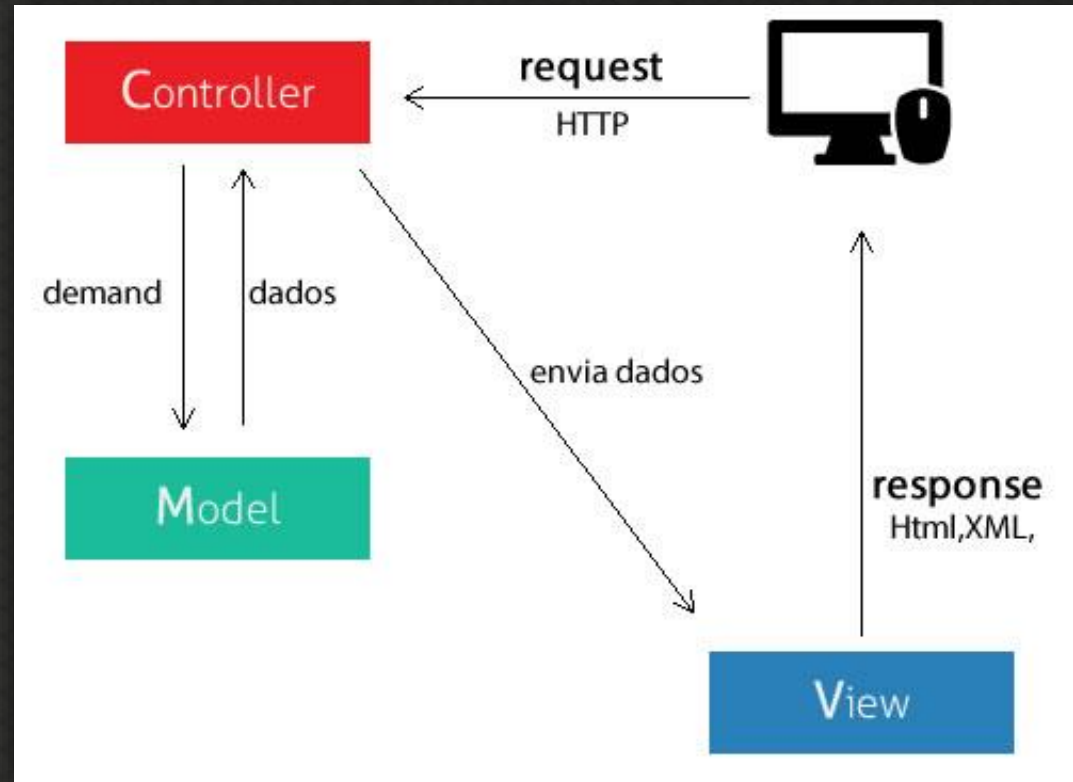
Arquitetura MVC

A arquitetura MVC é nada mais que um padrão de arquitetura de software, separando sua aplicação em **3 camadas**.

A camada de interação do usuário (**view**), a camada de manipulação dos dados (**model**) e a camada de controle (**controller**).

O **Controller**, por exemplo, é responsável por receber todas as requisições do usuário.

Seus métodos chamados *actions* são responsáveis por uma página, controlando qual **model** usar e qual **view** será mostrado ao usuário.



A imagem representa o fluxo do MVC em um contexto de Internet, com uma requisição HTTP e resposta em formato HTML.

JS

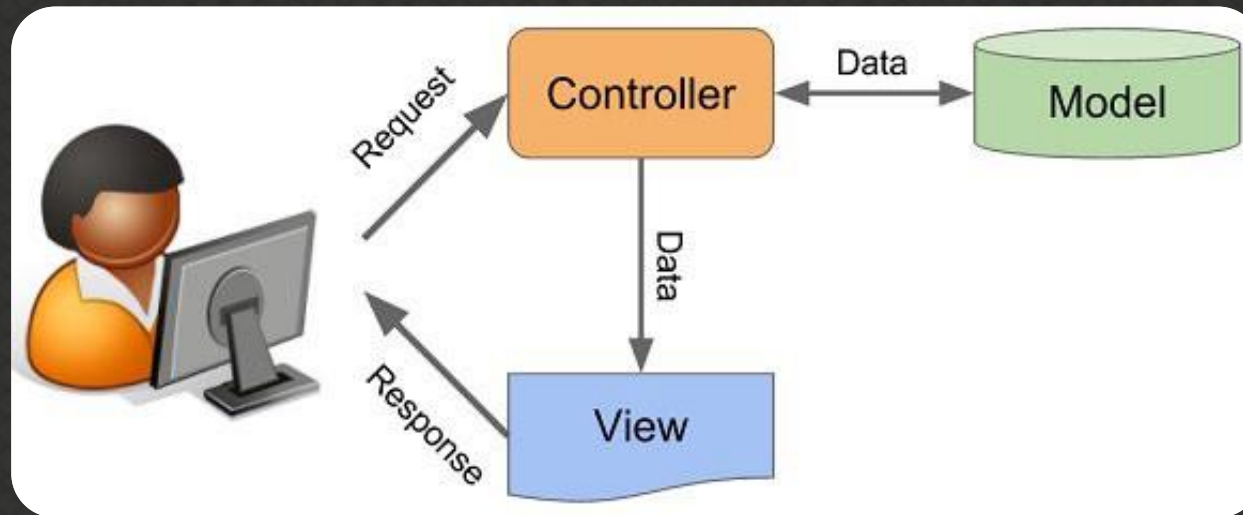
Arquitetura MVC

Resumindo a arquitetura MVC temos:

Models: responsável pela manipulação dos dados;

Views: responsável pela interação com o usuário;

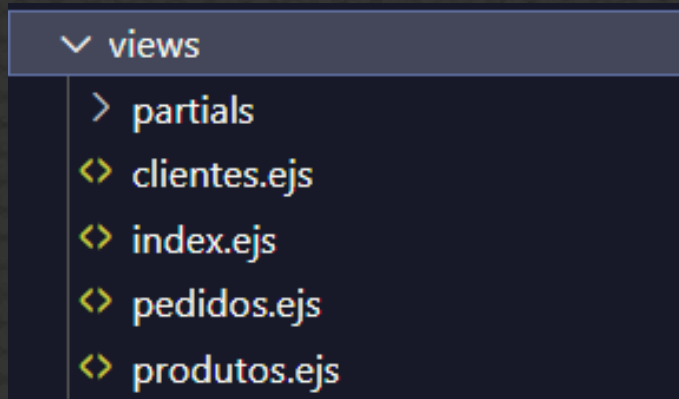
Controllers: camadas de controle.



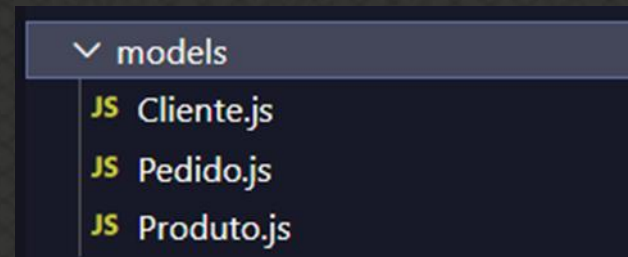
Arquitetura MVC

Aqui temos um exemplo real da arquitetura MVC sendo aplicada em um projeto. Temos uma pasta “**views**” onde estão armazenadas as páginas que o usuário terá contato. A pasta “**models**” onde estarão as regras de negócio e os códigos responsáveis pela manipulação de dados. E a pasta “**controllers**” onde estarão os arquivos de controle responsáveis por tratar as requisições dos usuários.

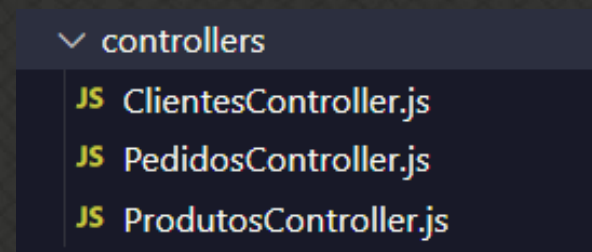
Views



Models



Controllers



JS

Arquitetura MVC

Neste exemplo, no arquivo “PedidosControllers.js” incluímos os códigos com as rotas referentes a entidade “Pedido”. Note também que foi necessário **importar** o “**express.Router()**”, responsável por gerenciar as rotas e também realizar sua exportação.

```
import express from "express"
const router = express.Router()

// ROTA PEDIDOS
router.get("/pedidos",function(req,res){
  const pedidos = [
    {numero: "983721931", valor: 1200},
    {numero: "983721932", valor: 900},
    {numero: "983721933", valor: 3200},
    {numero: "983721934", valor: 150}
  ]
  res.render("pedidos", {
    pedidos: pedidos
  })
})

export default router
```



JS

Arquitetura MVC

Feito isso, importamos todos os **controllers** no arquivo principal "**index.js**".

```
import express from "express" // Importando o Express
const app = express() // Iniciando o Express
// Importando os Controllers (onde estão as rotas)
import ClientesController from "../controllers/ClientesController.js"
import PedidosController from "../controllers/PedidosController.js"
import ProdutosController from "../controllers/ProdutosController.js"
```

Ainda no arquivo "**index.js**", definimos o uso das rotas que estão nos Controllers:

```
// Definindo o uso das rotas dos Controllers
app.use("/", ClientesController)
app.use("/", PedidosController)
app.use("/", ProdutosController)
```

Assim, as rotas que antes estavam no arquivo "**index.js**" agora se localizam devidamente organizadas em seus arquivos de **controllers**.



JS

Common JS x ES6 Modules



A partir de agora para trabalharmos com **módulos** utilizaremos o modo de importação do **EcmaScript 6 (ES6)**. Ele difere do modo de **importação** de módulos do **CommonJS** que é nativo do Node. Veja a diferença:

Common JS :

```
const express = require("express") // Importando o Express
const app = express() // Iniciando o Express
```

ES6 Modules:

```
import express from 'express' // Importando o Express
const app = express() // Iniciando o Express
```

JS

Common JS x ES6 Modules



Há mudanças também na forma de **exportar** módulos.

Veja a diferença:

Common JS :

```
module.exports = router // exportando o módulo "router"
```

ES6 Modules:

```
export default router //exportando o módulo "router"
```

JS

ES6 Modules

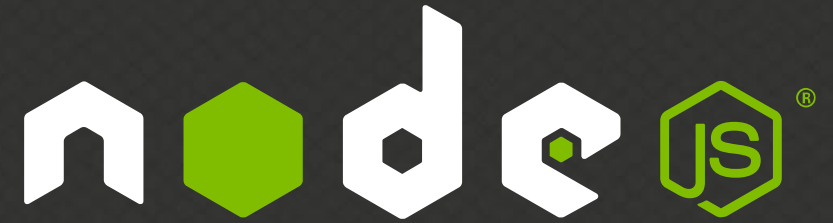


Para que a mudança funcione, precisamos informar no arquivo **package.json** do projeto que estaremos utilizando os módulos do ES6, conforme a seguir:

```
{  
  "name": "loja",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "author": "",  
  "license": "ISC",  
  "dependencies": {  
    "ejs": "^3.1.9",  
    "express": "^4.18.2",  
    "nodemon": "^3.0.1"  
  },  
  "type": "module"  
}
```

Incluir essa linha no arquivo package.json do projeto:

JS



Aula 05:

Arquitetura MVC